Deployment Report

Smart Hydroponic Lettuce System

AI-Driven Optimization System for Hydroponic Lettuce

Prepared for the Final Year Research Project deployment planning stage

Project: AI-Driven Optimization System for Hydroponic Lettuce

Project ID: 25-26J-493

Deployment Model: Cloud-hosted microservices + mobile APK

Recommended Platform Stack: AWS-style architecture / managed deployment

March 2026

Contents

1. Introduction 3
2. System Overview 3
3. Component Deployment Mapping 4
4. Proposed Cloud Deployment Architecture 5
   4.1 Compute Layer 5
   4.2 Database Layer 5
   4.3 Storage Layer 5
   4.4 Networking Layer 6
   4.5 Domain and Security 6
   4.6 High-Level Deployment View 6
5. Data, Storage, and Networking Architecture 7
6. Deployment Workflow 7
7. Mobile APK Build and Release Plan 8
8. Security, Reliability, and Monitoring 8
9. Advantages of the Proposed Deployment 8
10. Conclusion 9

1. Introduction

This report presents a suitable deployment plan for the Smart Hydroponic Lettuce System, a multi-component research project that combines mobile computing, artificial intelligence, image processing, sensor ingestion, and cloud-hosted microservices. The proposed deployment is designed for a real demo environment first, while also remaining suitable for future production-scale extension.

Unlike a typical web-only application, this project contains a mobile frontend, multiple independent AI services, image upload and annotation flows, IoT sensor integration, and a centralized database. Therefore, the deployment approach must support modular hosting, shared persistence, secure public API access, and smooth Android application delivery.

2. System Overview

The Smart Hydroponic Lettuce System consists of a mobile application connected to multiple cloud-hosted backend services. Each service is responsible for a different research component while sharing user access control and common infrastructure.

- Authentication Service for registration, login, and JWT-based access validation.
- Weight Estimation and Growth Forecasting Service for image-based weight estimation and prediction logging.
- Disease / Leaf Health Service for leaf classification, tipburn detection, health scoring, and report history.
- Spoilage / Shelf-life Service for spoilage stage classification and remaining shelf-life prediction.
- Water Quality and Algae Warning Service for sensor ingestion, water health scoring, and algae-related alerting.
- React Native Expo mobile application that acts as the user-facing interface for all four components.

3. Component Deployment Mapping

The following table maps each project module to its major responsibilities and deployment considerations.

Component	Main Responsibility	Key Inputs / Outputs	Deployment Notes
Authentication Service	User registration, login, token generation, role validation	Input: credentials; Output: JWT, user profile	Deploy as independent FastAPI service; shared with all other components.
Weight & Growth Service	Weight estimation, plant scan storage, growth prediction logs	Input: plant images, sensor data; Output: estimates, forecasts	Requires PostgreSQL and optional image storage.
Disease / Leaf Health Service	Disease classification, tipburn detection, annotated images, saved reports	Input: leaf image; Output: class probabilities, tipburn areas, health report	Requires durable image/object storage and report history endpoints.
Spoilage Service	Shelf-life classification and spoilage stage analysis	Input: spoilage image / context data; Output: stage, remaining shelf life	ML service should be isolated to simplify scaling.
Water Quality Service	Sensor ingestion and water-quality warning generation	Input: IoT readings; Output: status, alerts, logs	Must expose API endpoints for ESP32 / simulator ingestion.
Mobile Frontend	Unified user experience across all modules	Input: public backend APIs; Output: mobile dashboard, scan screens, history screens	Built as Android APK via Expo EAS Build.

4. Proposed Cloud Deployment Architecture

The most suitable architecture for this project is a cloud-hosted microservice model with a centralized PostgreSQL database and durable object storage. An AWS-style layout is recommended for documentation and future production use, while the same structure can also be implemented on managed platforms such as Railway or Render during the development and demo stage.

4.1 Compute Layer

Each backend service should be deployed independently so that failures, updates, or resource-heavy inference workloads in one component do not disrupt the others.

- Each service runs as an independent FastAPI application.
- Services can be deployed on Amazon EC2 or containerized using Docker on ECS / Fargate.
- Authentication remains lightweight, while ML-heavy services remain isolated and scalable.
- Each service should expose a public HTTPS endpoint used by the mobile application.

4.2 Database Layer

A single managed PostgreSQL instance should be used as the primary database for the full project. This approach is cleaner and more reliable than keeping SQLite in production, especially because multiple services and multiple team members must access shared data consistently.

- Use PostgreSQL as the main production database.
- Keep component data separated using dedicated tables per service.
- Store users, logs, scans, plant metadata, predictions, and sensor readings centrally.
- Use connection pooling and environment-variable-based DATABASE_URL configuration.

4.3 Storage Layer

Cloud object storage is required for uploaded plant images, scan archives, and annotated outputs. Local upload folders are not reliable in cloud deployments because files can be lost during redeployment or instance restart.

- Use Amazon S3 for uploaded scan images and annotated disease outputs.
- Store disease images, spoilage images, and optional generated report assets in object storage.
- Persist only the file path / object key in PostgreSQL, not binary files in database tables.

4.4 Networking Layer

Secure communication between the mobile application, backend services, and database must be managed through cloud networking controls.

- Use a Virtual Private Cloud (VPC) to isolate database and internal service communication.
- Use Security Groups to control inbound and outbound traffic.
- Expose only required service ports publicly.
- Keep the database private and reachable only from approved backend services.

4.5 Domain and Security

Public-facing endpoints should be protected with HTTPS and organized using dedicated subdomains. This is especially useful when the mobile app needs clean and stable base URLs for multiple services.

- Use Route 53 for domain and subdomain management.
- Use AWS Certificate Manager for SSL certificates.
- Expose service URLs such as auth, weight, disease, spoilage, and water under separate subdomains.
- Use JWT tokens for secure authentication across all mobile requests.

4.6 High-Level Deployment View

Android Mobile App (React Native Expo APK)
Public API Layer: Auth Service | Weight Service | Disease Service | Spoilage Service | Water Quality Service
Core Data Layer: Managed PostgreSQL
Object Storage Layer: Amazon S3 for scan images and annotated outputs

5. Data, Storage, and Networking Architecture

The deployment must support both structured application data and large unstructured scan files. Therefore, responsibilities should be separated clearly between the database and object storage.

Layer	Stores	Reason for Use
PostgreSQL	User accounts, plant metadata, prediction logs, disease reports, spoilage logs, water-quality readings	Best for structured relational data and multi-service consistency.
Amazon S3	Uploaded images, scan archives, annotated images, optional generated artifacts	Best for durable, scalable, file-based storage.
Networking Security	Private DB access, service-to-service communication, HTTPS public access	Protects services and prevents direct exposure of internal systems.

6. Deployment Workflow

A suitable deployment workflow for the hydroponic system is shown below.

1. The user installs and opens the Android mobile application.
2. The app connects to the Authentication Service and signs in the user securely.
3. The user selects one of the system modules from the dashboard.
4. If the user performs a scan, the mobile app uploads images to the relevant backend service.
5. If IoT readings are generated, the ESP32 or simulator sends sensor data to the Water Quality Service.
6. The selected ML service performs inference and writes prediction results to PostgreSQL.
7. Uploaded images and annotated outputs are saved to object storage.

8. The mobile app fetches reports, histories, alerts, and dashboard summaries from the relevant service.

9. The user can then review predictions, history logs, and warning outputs through the app interface.

7. Mobile APK Build and Release Plan

The frontend should be distributed as an Android APK using Expo EAS Build. Before building the APK, all localhost and laptop IP references must be replaced with hosted backend URLs.

- Configure production environment variables for all backend base URLs.
- Test login, scan, save-log, dashboard, history, and annotation flows using hosted endpoints on a real phone.
- Use Expo EAS Build to generate an installable APK for demos and user testing.
- Keep a separate production build profile and a preview/demo profile if needed.

8. Security, Reliability, and Monitoring

Because the project combines user accounts, plant data, AI results, and uploaded images, production deployment must emphasize security and operational reliability.

- Use JWT for authenticated requests from the mobile app.
- Restrict database exposure and store all secrets as environment variables.
- Use HTTPS for all public APIs.
- Back up PostgreSQL regularly and keep storage lifecycle policies for archived scans.
- Monitor service health using /health endpoints and platform logs.
- Keep model files versioned so that service redeployments remain reproducible.

9. Advantages of the Proposed Deployment

The proposed deployment offers several advantages for both the final-year project demo and any future real-world extension.

- Scalability: each backend service can scale independently based on workload.
- Reliability: one failing component does not automatically take down the whole system.

- Maintainability: each module can be developed, tested, and redeployed independently.
- Security: central identity control, HTTPS, and private database networking improve protection of user and plant data.
- Practicality: the architecture supports both AI image analysis and IoT sensor ingestion within one mobile system.

10. Conclusion

The Smart Hydroponic Lettuce System requires a deployment design that is more advanced than a simple web application because it combines mobile delivery, AI image inference, multiple backend services, database persistence, and sensor-driven workflows. A cloud-hosted microservice architecture with PostgreSQL, object storage, secure public APIs, and Android APK delivery is the most suitable deployment approach for this project.